

Rapport TP2 : MongoDB Agrégations et index

Partie 2 — Analyse des performances avec `explain()`

Tableau comparatif (Exemple de valeurs théoriques)

Note : Les valeurs réelles dépendront du jeu de données inséré, mais le comportement sera similaire.

Métrique d' <code>executionStats</code>	SANS Index (Collection Scan)	AVEC Index <code>{ statut: 1, date: -1 }</code> (Index Scan)
<code>winningPlan.stage</code>	COLLSCAN	FETCH > IXSCAN
<code>totalDocsExamined</code>	100 000 (Toute la collection)	40 000 (Seulement les statuts "livree")
<code>executionTimeMillis</code>	~ 45 ms	~ 5 ms

Analyse : Sans l'index, MongoDB doit scanner la collection complète (COLLSCAN) pour trouver tous les documents correspondant au filtre `statut: "livree"`, ce qui est très inefficace. Avec l'index, il réalise un IXSCAN (scan de l'index) et ne récupère que les documents pertinents. `totalDocsExamined` chute drastiquement pour s'aligner sur `nReturned` (le nombre de documents matchant), ce qui fait s'effondrer le temps d'exécution (`executionTimeMillis`).

Quand faut-il éviter de trop indexer ?

Il faut éviter de trop indexer (sur-indexation) car bien que les index accélèrent la lecture (`find`, `aggregate`), ils **ralentissent l'écriture** (`insert`, `update`, `delete`), puisque MongoDB doit mettre à jour chaque index à chaque modification de la collection. De plus, chaque index consomme de la **mémoire RAM** et du stockage disque, ce qui peut saturer la

machine si des index inutiles sont conservés. L'idéal est de créer uniquement les index qui ciblent les requêtes les plus fréquentes et les plus critiques.